

Auditoria Tecnica Integral

Proyecto: distribuidora-app

Fecha de revision: 15/04/2026 | Entorno revisado: React 19 + Vite + Supabase + TanStack Query + Dexie

Alcance

La auditoria cubrio arquitectura frontend, flujos funcionales principales, manejo offline, integraciones externas, seguridad practica, escalabilidad y experiencia operativa. No se modifico codigo de la app; solo se relevo, verifiko y documento el estado actual.

Resumen Ejecutivo

- La app tiene una base tecnica mejor que el promedio: el proyecto compila, testea y construye bien, y no encuentre vulnerabilidades productivas conocidas en dependencias.
- El mayor problema no esta en la infraestructura sino en la coherencia operativa del dominio de pedidos: offline, sincronizacion, pagos parciales y feedback de UI no estan alineados.
- Si hoy hubiera que priorizar con criterio de negocio, trataria como bloqueadores F1 a F7 antes de seguir agregando funciones nuevas.
- No recomendaria llamar a esta version de primer nivel hasta endurecer esos flujos y simplificar la captura de pedidos.

Eje	Estado	Lectura
Base tecnica	Verde	Build, typecheck y tests pasan
Pedidos y offline	Rojo	Es el subsistema con mayor riesgo operativo
Seguridad practica	Amarillo	Sin CVEs productivos, pero con exposiciones evitables
Escalabilidad	Amarillo	Puede crecer, pero necesita ordenar consultas, lotes y modulos
UX operativa	Amarillo	Tiene potencia funcional, pero hoy exige demasiado contexto por pantalla

Metodologia

- Lectura dirigida del codigo fuente, con foco en autenticacion, pedidos, pagos, offline, sincronizacion, compras e integraciones.
- Revision de configuracion de build, lint, seguridad, migraciones y estructura modular.
- Ejecucion de verificaciones objetivas sobre el repo auditado el 15/04/2026.
- Cruce entre hallazgos funcionales, riesgo operativo y deuda estructural para priorizar impacto real.

Chequeo	Resultado
npm run typecheck	OK
npm run test:run	OK (27 archivos, 522 tests)
npx eslint src --ext .js,.jsx,.ts,.tsx	OK
npm run lint	Timeout a los 94s en este entorno

npm run build	OK; warning por chunks > 500 kB
npm audit --omit=dev --json	0 vulnerabilidades productivas

Fortalezas Reales Detectadas

- La base tecnica es moderna y coherente: React 19, Vite, TanStack Query, Supabase y Dexie para offline.
- La cobertura automatizada es amplia para un proyecto de este tipo: 27 archivos de test y 522 tests pasaron en esta auditoria.
- Typecheck, build de produccion y audit de dependencias productivas pasaron correctamente.
- Hay evidencia de una cultura de seguridad y observabilidad: RLS y migraciones recientes, Sentry y sanitizacion ya introducida en parte del codigo.
- El proyecto ya hizo trabajo previo de code splitting y de encapsulacion de servicios; eso acelera una mejora seria si se prioriza bien.

Hallazgos Priorizados

Ordene los hallazgos por impacto tecnico y operativo. Los P1 son los que mas probablemente generen errores de negocio, perdida silenciosa de datos o comportamiento inconsistente para el usuario.

F1 - La deduplicacion de operaciones offline esta rota (P1)

Categoria: Bug / Offline / Logica

El hash usado para detectar duplicados incorpora Date.now(), asi que dos operaciones identicas nunca producen el mismo valor.

Impacto: La cola offline puede aceptar pedidos o mermas duplicadas. El codigo promete idempotencia, pero en la practica no la cumple.

Recomendacion: Separar hash de contenido e identificador unico. El hash de contenido debe ser determinista y usarse para deduplicar; el id unico puede seguir teniendo timestamp si hace falta.

Evidencia:

```
src/lib/offlineDb.ts:138-147 generateOperationHash agrega Date.now() al hash
src/lib/offlineDb.ts:159-166 la deteccion de duplicados compara igualdad exacta sobre ese hash
```

F2 - Limpiar pedidos offline no limpia la cola real (P1)

Categoria: Bug / Offline / Consistencia

La UI vacia los arrays locales, pero la limpieza real solo borra operaciones con estado completed.

Impacto: Operaciones pending o failed quedan en IndexedDB. El usuario puede creer que limpio todo y luego ver reaparecer pendientes o disparar una sincronizacion no deseada.

Recomendacion: Agregar una accion explicita para purge de la cola o permitir borrar por estado. La UI de limpiar debe reflejar exactamente lo que pasa en almacenamiento.

Evidencia:

```
src/hooks/useOfflineSync.ts:614-620 limpiarPedidosOffline llama cleanupOldOperations(0)
src/lib/offlineDb.ts:283-294 cleanupOldOperations elimina solo status == completed
```

F3 - Se informa exito en offline aunque la persistencia falle (P1)

Categoria: Bug / Offline / Perdida silenciosa

El guardado en cola es asincronico y no bloqueante; si falla, igual se actualiza la UI y se devuelve success.

Impacto: Puede haber pedidos o mermas que parecen guardados, pero nunca quedaron persistidos y por lo tanto nunca van a sincronizar.

Recomendacion: Esperar confirmacion minima de persistencia o marcar el item como pending_local hasta que IndexedDB confirme. Si falla, la UI no deberia reportar guardado correcto.

Evidencia:

```
src/hooks/useOfflineSync.ts:328-348 queueOperation se ejecuta en segundo plano y solo loguea el error
src/hooks/useOfflineSync.ts:350-353 la UI agrega el pedido y retorna success: true antes de confirmar persistencia
src/hooks/useOfflineSync.ts:369-394 repite el mismo patron para mermas
```

F4 - El auto-sync no arranca si la app inicia online y los pendientes llegan despues (P1)

Categoria: Bug / Offline / Sincronizacion

El effect que dispara sincronizacion depende solo de isOnline. Si los arrays de pendientes se cargan mas tarde, el effect no vuelve a correr.

Impacto: La app puede quedar online con operaciones en cola sin sincronizarlas hasta que el usuario fuerce una accion manual o cambie el estado de conexion.

Recomendacion: Agregar dependencias de pendientes o un gatillo derivado como pendingCount. Idealmente, sincronizar al terminar loadPendingOperations si hay conectividad.

Evidencia:

```
src/hooks/useSyncManager.ts:151-157 useEffect depende solo de [isOnline]
```

F5 - El flujo nuevo de pedidos aparenta soportar offline, pero en realidad no lo hace (P1)

Categoria: Bug / UX / Offline

PedidosContainer pasa isOffline al modal, pero el modal ni siquiera consume esa prop; al guardar siempre ejecuta la mutacion online.

Impacto: Hay una falsa sensacion de soporte offline. El usuario puede abrir el modal estando sin red y encontrarse con una falla en el momento mas sensible del flujo.

Recomendacion: Unificar de verdad el flujo online/offline. Si el soporte offline sigue existiendo, el modal debe derivar al guardado offline; si no, quitar la se?al de UI para no inducir a error.

Evidencia:

```
src/components/containers/PedidosContainer.tsx:589-603 guardar pedido llama siempre  
crearPedido.mutateAsync(...)  
src/components/containers/PedidosContainer.tsx:824-835 se pasa isOffline={!isOnline} al modal  
src/components/modals/ModalPedido.tsx:95-96 la prop isOffline existe en la interfaz  
src/components/modals/ModalPedido.tsx:100-122 la deestructuracion de props no incluye isOffline
```

F6 - El pago parcial se captura en la UI pero el alta de pedido lo ignora (P1)

Categoria: Bug / Negocio / Pagos

La pantalla permite cargar montoPagado cuando el estado es parcial, pero la mutacion crearPedido no usa ese dato ni registra el pago.

Impacto: Riesgo financiero directo: el usuario cree que registro un anticipo, pero el backend deja el pedido como pendiente o sin reflejar el monto realmente cobrado.

Recomendacion: Cerrar la paridad funcional del flujo nuevo con el legado: o el RPC debe aceptar montoPagado, o debe existir una segunda operacion atomica para registrar el pago parcial.

Evidencia:

```
src/components/modals/ModalPedido.tsx:615-637 la UI solicita monto del pago parcial  
src/components/containers/PedidosContainer.tsx:589-603 el contenedor envia montoPagado en el  
payload  
src/hooks/queries/usePedidosQuery.ts:40-54 CrearPedidoInput declara montoPagado  
src/hooks/queries/usePedidosQuery.ts:278-304 crearPedido no usa montoPagado ni llama a un  
registro de pago
```

F7 - La sincronizacion offline pierde campos de negocio (P1)

Categoria: Bug / Logica / Offline

El adaptador offline conserva montoPagado en memoria, pero el contrato CrearPedidoFunction y la llamada real no pasan ese valor ni otros campos relevantes del pedido.

Impacto: Pedidos creados offline pueden sincronizarse con datos incompletos o degradados: pagos parciales, tipo de factura y fechas operativas pueden perderse o caer en defaults.

Recomendacion: Alinear el contrato offline con el contrato online real. El payload persistido debe ser canonico y el sincronizador no deberia descartar atributos del negocio.

Evidencia:

```
src/hooks/useOfflineSync.ts:91-101 CrearPedidoFunction solo acepta hasta estadoPago
src/hooks/useOfflineSync.ts:138-149 operationToPedidoOffline si conserva montoPagado
src/hooks/useOfflineSync.ts:523-531 la sincronizacion llama crearPedidoFn sin montoPagado,
tipoFactura ni fechaEntregaProgramada
```

F8 - La busqueda de pedidos interpola texto sin sanitizar en filtros PostgREST (P2)

Categoría: Seguridad / Robustez / Datos

El termino de busqueda se inserta directo dentro del string de .or(...). En otras partes del proyecto ya existe un helper de escape, pero aqui no se usa.

Impacto: Un termino con caracteres especiales puede romper consultas, generar resultados inesperados o abrir una superficie de inyeccion en filtros PostgREST.

Recomendación: Aplicar el helper de sanitizacion de forma centralizada en todos los builders de filtros y cubrirlo con tests de caracteres especiales.

Evidencia:

```
src/hooks/queries/usePedidosQuery.ts:228-235 query.or(...) usa search trimmeado sin escape
src/components/containers/PedidosContainer.tsx:373-378 export y fetch total repiten el patron sin sanitizar
src/services/api/clienteService.ts:6 y 76-80 muestran que ya existe escapePostgresFilter para este problema
```

F9 - La API key de Google queda expuesta en el browser y ademas se reenvia al webhook (P2)

Categoría: Seguridad / Secretos / Integraciones

El cliente lee VITE_GOOGLE_API_KEY y la manda en el body al webhook de n8n.

Impacto: La clave queda visible en el frontend y tambien en logs, trazas o terceros intermedios del webhook. Es una mala frontera de confianza para una credencial de infraestructura.

Recomendación: Mover toda llamada a Google Routes al backend o a n8n con secreto solo server-side. El frontend deberia enviar solo ids y coordenadas necesarias.

Evidencia:

```
src/hooks/useOptimizarRuta.ts:63-66 la key se toma de import.meta.env
src/hooks/useOptimizarRuta.ts:176-182 google_api_key viaja en el requestBody
src/hooks/useOptimizarRutaPreventista.ts:108-117 repite el mismo patron para preventistas
```

F10 - El escaneo de facturas depende de URLs publicas (P2)

Categoría: Seguridad / Privacidad / Storage

La imagen de factura se sube a Storage y luego se obtiene un getPublicUrl para enviarlo al webhook.

Impacto: Si el bucket es publico o su politica cambia, documentos sensibles pueden quedar accesibles por URL. Para comprobantes y datos fiscales es una exposicion evitable.

Recomendación: Usar bucket privado y signed URLs de corta vida, o mejor aun procesar el archivo por backend sin publicarlo.

Evidencia:

```
src/components/modals/ModalCompra.tsx:412-446 flujo de upload y envio al webhook
src/components/modals/ModalCompra.tsx:438-439 se usa getPublicUrl(fileName)
```

F11 - Las operaciones masivas no son atomicas y varias son seriales (P2)

Categoría: Escalado / Consistencia / Operaciones masivas

Hay lotes que actualizan registro por registro o registran historial en best-effort, lo que deja el sistema expuesto a estados parciales.

Impacto: Con mas volumen o mas latencia, aumenta la chance de medias actualizaciones, inconsistencias de auditoria y tiempos largos de espera para el operador.

Recomendacion: Llevar estas acciones a RPCs o endpoints transaccionales que resuelvan lote completo, historial y reglas de negocio en una sola unidad atomica.

Evidencia:

```
src/hooks/queries/usePedidosQuery.ts:628-650 entrega masiva actualiza y luego inserta historial sin transaccion
src/hooks/queries/usePedidosQuery.ts:756-789 pagos masivos actualiza pedido por pedido en un loop
src/components/containers/PedidosContainer.tsx:341-349 asignacion masiva itera con await secuencial
```

F12 - Persisten varias consultas sin paginacion y con select * (P2)

Categoria: Escalado / Rendimiento / Datos

Todavia hay hooks que traen tablas completas y todas las columnas aunque la pantalla no necesite todo.

Impacto: Con mas clientes, productos o pedidos, crecen el payload, el tiempo de render y la memoria consumida. Hoy puede ser tolerable; a mediano plazo se vuelve friccion diaria.

Recomendacion: Definir proyecciones minimas por vista, paginacion server-side y fetch especificos para exportes o modales pesados.

Evidencia:

```
src/hooks/queries/useClientesQuery.ts:21-28 fetchClientes hace select(*) sin paginacion
src/hooks/queries/useClientesQuery.ts:42-50 fetchClientesByZona repite select(*) completo
src/components/containers/PedidosContainer.tsx:356-378 el export reconsulta todos los pedidos filtrados sin paginacion
```

F13 - Hay componentes y modales que concentran demasiadas responsabilidades (P2)

Categoria: Simplificacion / Arquitectura / UX

Los archivos mas grandes mezclan reglas de negocio, estado, validaciones, calculos fiscales, efectos y UI dentro del mismo bloque.

Impacto: Esto vuelve lenta cualquier mejora, incrementa regresiones y tambien complica la experiencia del usuario porque los flujos terminan resolviendose en modales gigantes con demasiadas decisiones juntas.

Recomendacion: Descomponer por caso de uso: captura del pedido, cobro, entrega y acciones masivas. Para UI, conviene usar pasos o paneles dedicados en lugar de un solo modal omnisciente.

Evidencia:

```
src/components/modals/ModalCompra.tsx 56937 bytes
src/components/containers/PedidosContainer.tsx 43718 bytes
src/components/modals/ModalPedido.tsx 37582 bytes
src/hooks/queries/usePedidosQuery.ts 25870 bytes
```

F14 - El bundle principal sigue pesado y el lint global no escala bien al repo (P3)

Categoria: Rendimiento / Build / DX

La build ya tiene manualChunks, pero aun aparecen trozos muy grandes. Ademas, eslint src corre bien, mientras que eslint . se vence en este entorno.

Impacto: Carga inicial mas costosa, features administrativas con costo alto de descarga y una experiencia de mantenimiento mas lenta para el equipo.

Recomendacion: Lazy load mas agresivo de exportes/reportes/modales pesados, aislar features de admin y ampliar ignores del lint para artefactos no fuente.

Evidencia:

```
vite build 15/04/2026: lib-excel 936.60 kB, lib-pdf 387.12 kB, index 366.90 kB, vendor-sentry 303.83 kB  
vite.config.js:181-233 ya existe una estrategia de manualChunks  
eslint.config.js:9-10 solo ignora dist y *.d.ts; npm run lint se vencio a los 94s, npx eslint src ... si completo
```

F15 - La base de testing es buena, pero hoy no cubre varias regresiones reales (P3)

Categoria: Calidad / Testing / Confiabilidad

Las pruebas pasan y eso es una fortaleza, pero no detectaron problemas criticos de parity entre el flujo nuevo y el offline. Ademas, el run emite varias advertencias de act(...) en tests de UI.

Impacto: El equipo puede tener una sensacion de seguridad mayor a la real. Algunas regresiones funcionales importantes estan fuera del radar actual de tests.

Recomendacion: Agregar pruebas de regresion para alta de pedido online/offline, pagos parciales, limpieza real de cola y autosync al iniciar online con pendientes.

Evidencia:

```
npm run test:run 15/04/2026: 27 archivos y 522 tests OK  
Durante el run aparecieron warnings repetidos de React act(...) en suites de hooks y UI  
Los bugs F1-F7 no fueron capturados por la suite actual
```

Lectura Por Eje

Bugs

Los bugs con impacto mas inmediato estan en F1, F2, F3, F4, F5, F6 y F7. No son detalles cosmeticos: afectan pedidos, pagos y persistencia offline.

Errores de logica

F1, F4, F6 y F7 muestran desacoples entre la intencion del sistema y su comportamiento real. El problema principal es la falta de un contrato unico entre flujo online, offline y sincronizacion.

Escalado

F11, F12 y F13 son la deuda que hoy no rompe, pero si va a castigar cuando suban volumen, usuarios o complejidad operativa. Las operaciones masivas y los select(*) son los primeros candidatos a refactor.

Optimizacion y rendimiento

F12 y F14 concentran la mejora mas clara: menos payload, mas code splitting y menos trabajo innecesario en cliente. Hay una buena base, pero todavia quedan chunks y pantallas pesadas.

Simplificacion de la app y UX

F13 es el eje central. La app tiene muchas funciones, pero varias conviven en modales y contenedores demasiado cargados. Conviene separar por tarea operativa y mostrar opciones avanzadas solo cuando agregan valor.

Seguridad

F8, F9 y F10 son las prioridades practicas. No vi evidencia de secretos commiteados ni de dependencias productivas vulnerables, pero si hay exposicion innecesaria de credenciales, URLs y superficies de consulta.

Simplificacion de Procesos y UX

- Separar el alta de pedido en pasos cortos: Cliente, Productos, Cobro y Entrega/Resumen. Hoy demasiadas decisiones conviven en un solo modal.
- Tratar el offline como estado de primer nivel: banner claro, cola visible, estados de persistencia y sincronizacion entendibles para el operador.
- Mover acciones avanzadas o de baja frecuencia fuera del flujo central: exportaciones, optimizacion de rutas, acciones masivas y escaneo de facturas no deberian cargar el camino principal de cada rol.
- Reducir el numero de modales gigantes. Para compras, pedidos y rutas conviene evaluar pantallas o drawers dedicados con estado propio y validaciones por etapa.
- Unificar pagos en un solo modelo visual y tecnico: pendiente, parcial y pagado, con saldo restante siempre visible y el mismo comportamiento online/offline.

Recomendaciones De Seguridad

- No enviar credenciales de terceros desde el frontend. Las integraciones con Google deben vivir detras de una capa server-side.
- Procesar facturas con bucket privado y signed URLs temporales, o mediante upload directo al backend sin pasar por URL publica.
- Considerar secureStorage como ofuscacion util, no como barrera fuerte. Si un atacante ejecuta JavaScript en el mismo origen, puede recuperar la clave y el dato.
- Estandarizar sanitizacion de filtros PostgREST y agregar tests especificos de caracteres especiales.

- Agregar pruebas de regresion sobre politicas RLS y Storage para evitar que cambios operativos abran buckets, tablas o RPCs mas de la cuenta.

Hoja De Ruta Recomendada

0 a 2 semanas

- Resolver F1-F7 y agregar sus pruebas de regresion.
- Sanitizar todas las busquedas PostgREST reutilizando el helper existente.
- Cambiar facturas a bucket privado con signed URLs o pipeline server-side.
- Definir si el soporte offline de pedidos queda como feature oficial o se retira de la UI hasta endurecerlo.

2 a 6 semanas

- Implementar RPCs para pagos masivos, entregas masivas y asignaciones masivas.
- Pagar vistas grandes y reemplazar select(*) por proyecciones especificas por pantalla.
- Dividir PedidosContainer, ModalPedido y ModalCompra en modulos por caso de uso.

6 a 10 semanas

- Rediseñar UX operativa por rol: preventista, transportista y administracion.
- Separar funciones pesadas de exportacion y reporting y cargarlas bajo demanda.
- Completar observabilidad funcional: metricas de cola offline, tasa de sync y fallas de integraciones externas.

Acciones Inmediatas Mas Rentables

1. Corregir en una sola iteracion F1 a F7: cola offline, autosync, paridad del flujo de pedido y pagos parciales.
2. Mover el uso de Google Routes y el procesamiento de facturas a un backend o webhook con secretos solo server-side.
3. Cerrar un set minimo de pruebas de regresion: crear pedido parcial, crear offline, sincronizar offline, limpiar cola y buscar con caracteres especiales.
4. Llevar las acciones masivas a RPCs o endpoints transaccionales.
5. Rediseñar el flujo de alta de pedido en pasos cortos y con validaciones por etapa, en lugar de seguir cargando un unico modal.

Conclusion

Mi lectura final es que la app esta bastante mas cerca de ser una muy buena base que de ser un sistema desordenado; hay señales claras de criterio tecnico, cobertura y trabajo serio. Pero hoy el subsistema de pedidos, pagos y offline todavia tiene inconsistencias suficientes como para frenar la calificacion de primer nivel. Si se corrigen los P1 y se ordena el flujo operativo principal, el proyecto puede quedar en un nivel muy superior sin necesidad de reescribirlo entero.

Cantidad de archivos fuente relevados en src: 262. Artefacto generado automaticamente el 15/04/2026 para uso interno del proyecto.